

SIKDS

Secure Institutional Knowledge & Distribution System

Software Requirements Specification

Version: 2.3

Date: March 1st, 2026

Prepared by: Aicha BRIHMOUCHE, Yassir CHERDOUH,
Abdelhak KADOUCI, Sofia LALAM,
Ahmed Fateh GUENDOZ

0 | Contents

1 Introduction 2

1.1 Purpose 2

1.2 Scope 2

2 System Overview 2

2.1 System Context 2

2.2 System Architecture (High-Level) 3

2.3 Permissions-Based Access Control Model 3

2.3.1 Initial Set of System Permissions 3

2.3.2 Role Definitions 4

3 Functional Requirements 5

3.1 Document Management 5

3.2 Document Tagging System 8

3.3 Email Notification System 9

3.4 Watermarking System 10

3.5 Document Filtering System 11

3.6 Retrieval-Augmented Generation (RAG) 11

3.7 User Management & Authentication 12

3.7.1 Authentication Strategy 12

3.8 Audit and Compliance 14

4 Non-Functional Requirements 14

4.1 Maintainability 15

5 Technical Architecture 15

5.1 Technology Stack 15

5.1.1 Core Platform (Laravel) 15

5.1.2 AI Integration (Laravel AI SDK) 15

5.1.3 Data Layer 15

5.1.4 Watermarking 16

5.2 System Architecture Diagram 16

5.3 Data Flow 16

5.3.1 Document Upload and Indexing Flow 16

5.3.2 Document Download Flow with Watermarking 17

6 Data Requirements 17

6.1 Database Schema (High-Level) 17

6.1.1 Core Tables 17

7 Security Requirements 18

7.1 Authorization Requirements 18

7.2 Data Protection Requirements 18

7.3 Application Security 18

8 Acceptance Criteria 19

9 Future Enhancements 19

Appendix A Watermarking Strategy & Technical Design 20

Appendix A.1 Shared Audit System 20

Appendix A.2 Option 1 - Document Canary Trapping 20

1 | Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document provides a complete description of the Secure Institutional Knowledge & Distribution System (SIKDS). It details the functional and non-functional requirements, technical architecture, and implementation approach for a centralized platform that enables secure distribution of official ministry documents to subordinate institutions (universities) with integrated Retrieval-Augmented Generation (RAG) capabilities.

1.2 Scope

In Scope:

- Secure document upload and storage system
- Multilanguage platform (primary in French, with Arabic and English support)
- User authentication and permissions-based access control
- Document watermarking (metadata-based and visible text overlay)
- Download tracking and audit logging
- Email notification system for document events
- Document vectorization and embedding pipeline
- RAG-powered semantic search and question-answering

2 | System Overview

2.1 System Context

SIKDS serves as a centralized hub for official ministry communications to universities. Currently, document distribution occurs through fragmented channels (email, physical delivery, ad-hoc uploads), leading to:

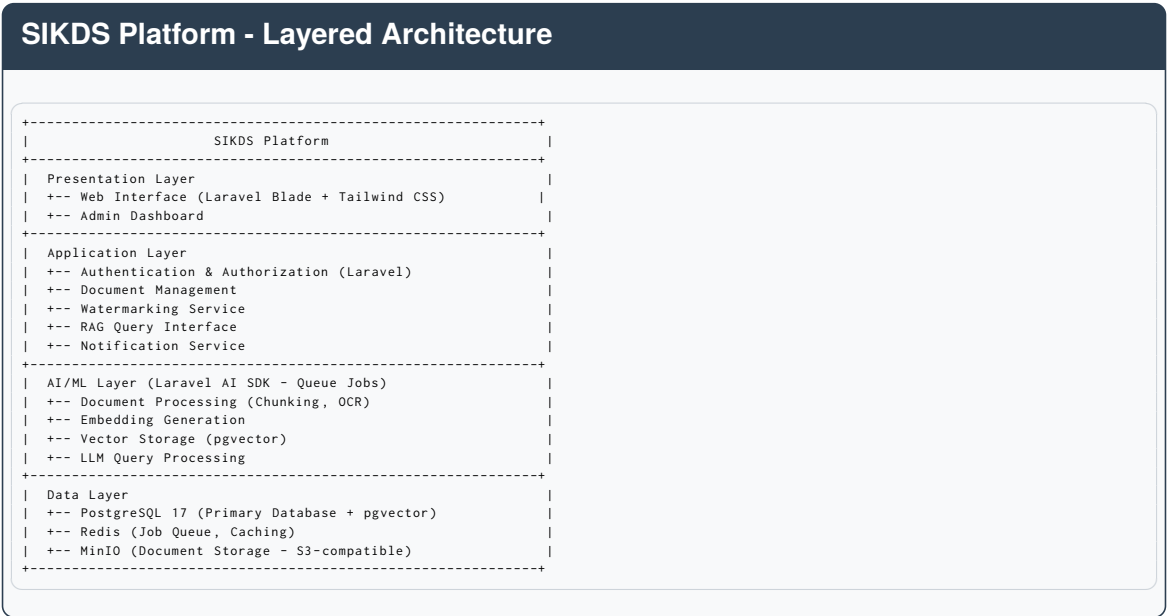
- Difficulty tracking who has received and accessed documents
- Inability to prevent unauthorized sharing
- Time-consuming manual search through archives
- Inconsistent interpretation of policy documents

SIKDS addresses these challenges through:

1. **Centralized Distribution:** Single source of truth for all official directives
2. **Accountability:** Watermarking and audit trails for every document access
3. **Intelligence:** RAG-powered semantic search eliminating manual archive searching

- 4. **Security:** Controlled access with comprehensive audit logging

2.2 System Architecture (High-Level)



2.3 Permissions-Based Access Control Model

SIKDS implements a flexible permissions-based access control system where users are characterized by their assigned set of permissions rather than pre-defined rigid roles.

2.3.1 Initial Set of System Permissions

The following permissions form the foundational access control building blocks:

Permission Code	Description
<i>Document Management</i>	
document.create	Upload new documents to the system
document.view.all	View all documents in the system regardless of restrictions
document.view.own_institution	View documents targeted to user's institution
document.view.assigned	View only documents explicitly assigned to user
document.edit	Edit document metadata and content
document.delete	Soft-delete documents (mark as inactive, visible only to Super-Admin)
document.restore	Restore a soft-deleted document to its previous state
document.publish	Publish draft documents making them visible to target audience

Permission Code	Description
<i>Document Distribution</i>	
distribution.manage	Define document target audiences and distribution rules
<i>Document Tagging</i>	
tag.assign	Assign tags to documents during upload
tag.manage	Create, edit, and delete custom tags
<i>User & Role Management</i>	
user.manage	Edit user profile, institution, department, and active status
user.deactivate	Deactivate user accounts (prevent login)
user.view.all	View all users in the system
user.assign.permissions	Assign permissions/roles to users
role.create	Create new custom roles by selecting permissions
role.edit	Modify existing role permissions
role.delete	Delete custom roles
role.view	View all roles and their permissions
<i>Institution Management</i>	
institution.create	Create new institutions (universities)
institution.edit	Edit institution details
institution.delete	Deactivate institutions
institution.view	View all institutions
<i>RAG & Search</i>	
rag.query	Submit queries to RAG system
search.basic	Use basic keyword search
<i>Audit</i>	
audit.view	View audit logs

Table 1: Initial System Permissions

2.3.2 Role Definitions

1. System Administrator (Super Admin) (*pre-defined; seeded on install*)

The sole role seeded by the system. Has all permissions enabled, providing complete system control.

Role Creation Workflow

Administrators create roles through the following process:

1. Access the “Role Management” interface
2. Click “Create New Role”
3. Enter role name and description
4. Select permissions from the checklist of available permissions
5. Save the role
6. Assign users to the newly created role

This workflow allows the system to adapt to any organizational structure without code changes.

3 | Functional Requirements

3.1 Document Management

Document Upload

Description:

Users with `document.create` permission shall be able to upload documents to the system.

Requirements:

- Accept PDF file format
- Maximum file size: 50MB per document
- Extract metadata: title, date, reference number, target audience
- Support batch upload (up to 5 documents simultaneously)
- Validate file integrity and scan for basic security issues

Document Metadata Management

Description:

System shall capture and store comprehensive metadata for each document.

Required Metadata Fields:

- Document Reference Number (auto-generated: YEAR-SEQUENCE)
- Title (required, max 200 characters)
- Issue Date (required)
- Effective Date (optional)

- Expiration Date (optional)
- Target Audience: `all` (every user), `specific_institutions`, or `specific_roles` — with a selector for the target entities
- Document Tags (see Section 3.2)
- Author/Department (auto-filled from uploader profile)
- Version Number (auto-incremented)

Acceptance Criteria:

- All required fields enforced at upload
- Reference numbers are unique and sequential
- Metadata searchable and filterable in admin interface

Document Versioning

Description:

System shall maintain version history for documents.

Requirements:

- When a document is updated, the previous version is automatically archived (status set to `Archived`)
- Archived versions are old, superseded versions of the document. They are **only viewable by administrators** (users with `document.view.all` permission) for audit and reference purposes; they are hidden from regular users.
- Each version has distinct version number (`v1`, `v2`)
- Archived versions are excluded from RAG indexing (only the current active version is indexed)

Document Lifecycle State Machine

Description:

Every document has a system-managed `status` field that governs its visibility, searchability, and behaviour. The status is **not** a user-assignable tag; it is controlled exclusively through system transitions.

Document States:

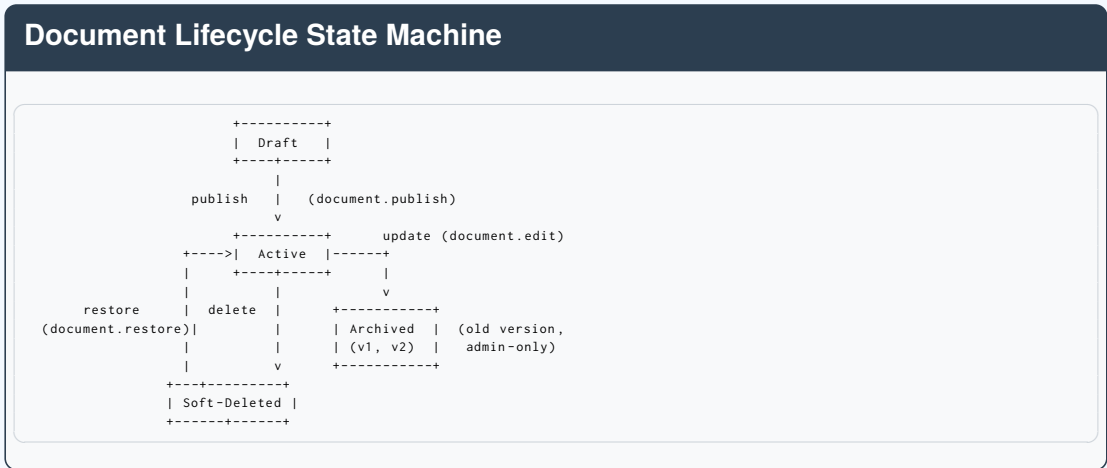
1. **Draft** - Document has been uploaded but not yet published. Visible only to users with `document.view.all` or the document creator. Not included in RAG index. Not downloadable by regular users.
2. **Active** - Document is published and visible to its target audience. Automatically indexed for RAG. Downloadable (with watermarking).
3. **Archived** - A previous version of a document that has been superseded by a

newer version. Visible **only to administrators** (users with `document.view.all`). Excluded from RAG index. Downloadable only by administrators.

4. **Soft-Deleted** - Document is logically removed from the system. Visible only to Super-Admin for review and restoration. Excluded from RAG index. Not downloadable. Data fully retained for potential restoration.

Valid State Transitions:

State Diagram:



Archive vs. Soft-Delete

Archived documents are *old versions* that have been superseded by a newer version of the same document. They remain in the system as historical records, viewable only by administrators. An archived version is created *automatically* when a document is updated - it is not a manual action.

Soft-Deleted documents are documents that have been *intentionally removed* from circulation (e.g., erroneous uploads, duplicates, or revoked directives). They are visible only to Super-Admin and can be restored.

Document Distribution Rules

Description:
System shall enforce document distribution rules based on target audience and user permissions.

Requirements:

- Documents can be targeted to: All, Specific Institutions, Specific Roles
- Users only see documents they are authorized to access based on:
 - Document target audience settings
 - User’s view permissions (`document.view.*`)

- User's institution
- Authorization checked at: list view, detail view, download, RAG query
- Email notifications sent when new document published to user's target group (SMTP configuration is provided)

3.2 Document Tagging System

Document Tags

Description:

System shall provide a flexible tagging system to categorize and organize documents by content type and subject matter.

Requirements:

Predefined Tag Categories:

The system includes the following predefined tags that users with `tag.assign` permission can apply to documents:

- **Document Type:** This is a suggested -non-exhaustive- list
 - Directive (Instruction officielle)
 - Decision (Décision)
 - Regulation (Règlement)
 - Report (Rapport)
- **Priority:**
 - Urgent
 - High
 - Normal
 - Low
- **Note:** Document status (Draft, Active, Archived, Soft-Deleted) is a **system-managed field**, not a tag. See the Document Lifecycle State Machine (Section 3.1) for details.

Custom Tags:

- Users with `tag.manage` permission can create, edit, and delete custom tags
- Custom tags must have: name, description, color code (for UI display)
- Unused tags can be deleted by users with `tag.manage` permission

Tag Assignment:

- Multiple tags can be assigned to a single document
- Tags are assigned during upload and can be removed later
- Tag changes are logged in audit trail
- Documents can be filtered and searched by tags

Acceptance Criteria:

- All predefined tags available on system initialization
- Custom tags can be created by authorized users
- Tags appear in document metadata and search filters
- Tag-based filtering returns correct document sets

3.3 Email Notification System

Notification Events and Content**Description:**

System shall send automated email notifications for document uploads to users in the document's target audience.

Requirements:**Notification Triggers:**

The system sends email notifications for the following events:

1. New Document Published

- Triggered when: Document published (status changed from Draft to Active)
- Recipients: All users in target audience
- Email contains:
 - Document title
 - Document reference number
 - Issue date
 - Document tags (Type, Priority)
 - Brief description/summary (if provided)
 - Direct link to view document

2. Document Updated

- Triggered when: Existing document modified with new version
- Recipients: All users in target audience
- Email contains:
 - Document title
 - Version number (old → new)
 - Change summary (if provided by uploader)
 - Document tags
 - Direct link to view new version

Email Format:

- All emails in French (primary) with Arabic and English translations available
- HTML format with ministry branding (logo, colors)
- Plain text fallback for email clients without HTML support
- Footer with system name and support contact

3.4 Watermarking System

Visible Watermarking

Description:

System shall apply a visible text watermark on every downloaded document page.

Requirements:**Watermark Composition:**

- Downloader information in small, semi-transparent text:
 - Recipient full name
 - Recipient Institution name
 - Download date and time
 - Unique download ID (UUID for traceability)

Metadata-Based Watermarking

Description:

System shall embed user-specific metadata invisibly in PDF file properties.

Requirements:

- Embed in PDF XMP metadata fields:
 - Recipient Name
 - Recipient Institution
 - Recipient Email
 - Download Timestamp
 - Unique Download ID (UUID)

3.5 Document Filtering System

Document Filtering & Sorting

Description:

The system shall provide a multi-layered filtering mechanism enabling users to locate specific documents efficiently. The filtering logic must integrate with the permissions-based access control model to ensure users only discover documents they are authorized to view.

Filtering Parameters:

Users shall be able to filter their authorized document pool by combining multiple parameters simultaneously:

- **Tags:** Filter by Document Type and Priority
- **Date Ranges:** Filter by bounded ranges for Issue Date or Effective Date (if set)
- **Search:** Filter by document Title (keyword match)

Sorting Capabilities:

Users shall be able to sort the filtered document results by the following criteria, in either ascending or descending order:

- Issue Date
- Effective Date (if any)
- Expiration Date (if any)

3.6 Retrieval-Augmented Generation (RAG)

Document Indexing Pipeline

Description:

System shall automatically process and index uploaded documents for semantic search.

Requirements:**Chunking Strategy:**

- Respect document structure (sections, paragraphs, articles)
- Preserve context: include section headers in chunks

Background Processing:

- Indexing runs asynchronously (Laravel Queue with Redis driver)
- All documents with status `Active` are automatically indexed; non-active documents (`Draft`, `Archived`, `Soft-Deleted`) are excluded
- Status tracking: `Pending` → `Processing` → `Indexed` → `Failed`

- Failed documents flagged for manual review
- Admin notified of indexing failures via dashboard

Semantic Query Interface

Description:

Users with `rag.query` permission shall be able to ask natural language questions about document corpus.

Requirements:**Query Input:**

- Text input accepting French, Arabic, or English natural language questions. As a start, the system will primarily support Arabic queries.
- Example queries: “*Quelles sont les dates limites d’inscription?*” (French), “*What are the enrollment deadlines?*” (English)

Retrieval Process:

- Convert query to embedding using multilingual model
- Resolve the set of document IDs the querying user is authorized to access, using the same authorization logic as the document list view (target audience, user permissions, user institution)
- Run vector similarity search in pgvector restricted to chunks whose parent document is in the authorized set
- Retrieve top 5–10 most relevant chunks
- Re-rank results by relevance

Response Generation:

- Send retrieved chunks + query to LLM (local model provided by ministry)
- Prompt engineering: “Answer ONLY based on provided context. Cite document names and sections.”
- Enforce citation requirements (every claim must reference source)

3.7 User Management & Authentication

3.7.1 Authentication Strategy

SSO Authentication Flow

Description:

SIKDS delegates authentication entirely to the Ministry SSO. Users with valid `@mesrs.dz` credentials who have been provisioned in SIKDS are granted access; all others are denied.

Flow:

1. User navigates to SIKDS platform and clicks on Login button.
2. SIKDS constructs a request (depending in the SSO type provided) and redirects the browser to the ministry SSO.
3. Ministry SSO authenticates the user then redirects back to SIKDS with a signed assertion.
4. SIKDS validates the token (signature, expiry, request ID) and extracts the email address (@mesrs.dz emails)
5. SIKDS looks up the email in the users table (`is_active = true`):
 - **Existing user:** Session created, user redirected to dashboard. Login event logged.
 - **First login:** Account auto-provisioned with profile data from SSO assertion (name, email, institution). Session created, login event logged.
 - **Deactivated:** Access denied with a message directing the user to contact the administrator. Denial event logged.

Fallback:

If the SSO server is unreachable, a clear error is shown and users are directed to contact IT.

Role and Permission Management

Description:

System shall provide flexible role creation and permission assignment capabilities.

Requirements:**Role Creation:**

- Users with `role.create` permission can define custom roles
- Role definition process:
 1. Enter role name and description
 2. Select permissions from checklist (grouped by category)
 3. Save role for assignment to users
- Roles can be edited by users with `role.edit` permission
- System prevents deletion of roles currently assigned to users

Permission Assignment:

- Users with `user.assign.permissions` can assign roles to users
- Multiple roles can be assigned to a single user (permissions are additive)
- Permission changes take effect immediately (no logout required)
- Audit log records all permission changes

Permission Validation:

- All actions check user permissions before execution
- Deny-by-default policy (explicit permission required)
- No privilege escalation possible through application
- Permission inheritance: Users automatically inherit permissions from all assigned roles

3.8 Audit and Compliance

Comprehensive Audit Logging

Description:

System shall log all security-relevant events in immutable audit trail.

Events to Log:

- User authentication (success/failure, IP address, timestamp)
- Document upload, view, download (with watermark ID)
- Document edits and version changes
- Tag assignments and changes
- User management actions (auto-provisioning, profile edits, deactivation)
- Role and permission changes
- Failed access attempts and permission denials
- Email notifications sent
- System configuration changes

Audit Log Protection:

- Logs stored in append-only database table
- Database triggers prevent UPDATE or DELETE operations

4 | Non-Functional Requirements

User Experience

- Consistent UI across all pages (Tailwind CSS for styling)
- Clear error messages with resolution guidance
- Responsive design

4.1 Maintainability

Code Quality
• Code documentation • Unit tests • Integration test coverage for critical user workflows
Documentation
• System administration guide

5 | Technical Architecture

5.1 Technology Stack

5.1.1 Core Platform (Laravel)

- **Framework:** Laravel 12.46.0 (PHP 8.3)
- **Frontend:** Laravel Blade templates + Tailwind CSS 3.4+ + Alpine.js 3.15
- **Components:**
 - Laravel Blade for templating (server-side rendering)
 - Laravel Eloquent ORM for database operations
 - Laravel Queue for background jobs (document indexing, email notifications)
 - Spatie Laravel Permission for permissions and role management

5.1.2 AI Integration (Laravel AI SDK)

- **Package:** Laravel AI SDK (Prism) for LLM provider abstraction
- **RAG Pipeline:** Implemented entirely as Laravel Queue jobs
- **Components:**
 - Document chunking and embedding generation via background jobs
 - pgvector similarity search via Laravel Eloquent

5.1.3 Data Layer

Primary Database: PostgreSQL 17

- **Extensions:**
 - pgvector 0.7 for vector similarity search

Cache & Queue: Redis 8.6

- **Usage:** Laravel cache; session storage; job queue; rate limiting

Object Storage: MinIO 0.14

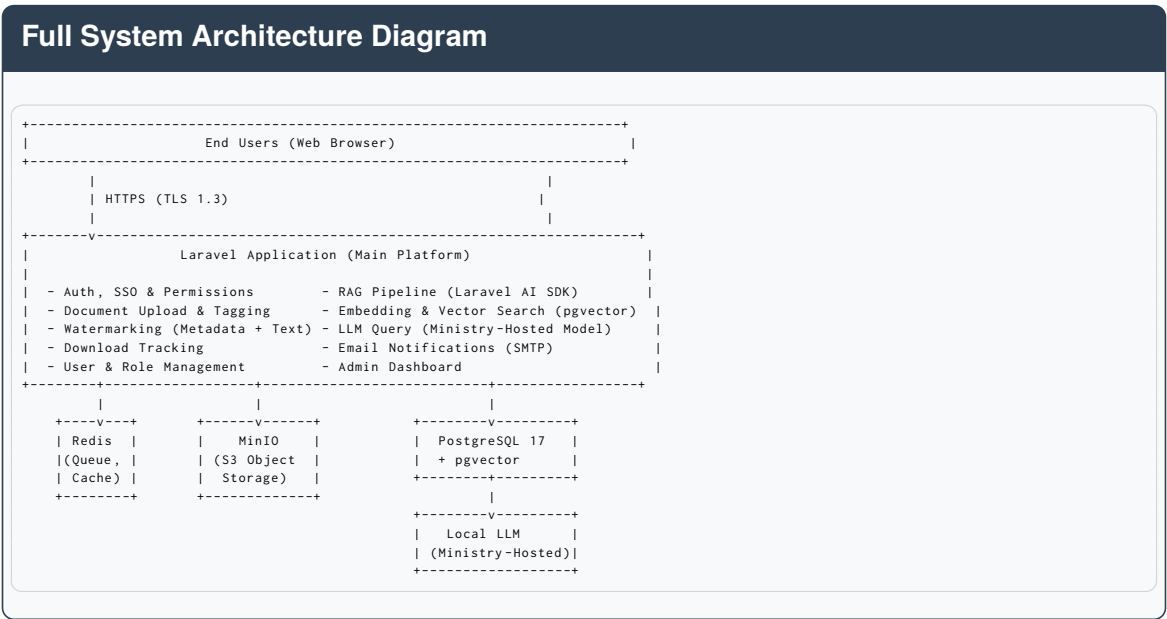
MinIO in Production

MinIO will be used throughout the project lifecycle, including production deployment.

5.1.4 Watermarking

- **Metadata Watermarking:** PHP PDF library (XMP metadata embedding)
- **Visible Text Watermark:** FPDF (PHP) - downloader info overlay on each page

5.2 System Architecture Diagram



5.3 Data Flow

5.3.1 Document Upload and Indexing Flow

1. **Upload Initiation:**
 - Ministry Admin selects file(s) via web interface
 - Laravel validates file type, size
 - Metadata stored in PostgreSQL
2. **Background Indexing (Async via Queue):**
 - Laravel dispatches job to Redis queue
 - Laravel queue worker picks up job
 - Extracts text
 - Chunks document using semantic-aware algorithm
 - Generates embeddings
 - Stores embeddings in pgvector with document/chunk references

- Updates document status: Processing → Indexed

3. Notification Dispatch:

- If document published (status = Active), trigger email notification
- Laravel queues SendNotificationJob with document details and tag information
- Email sent to all users in target audience

4. Error Handling:

- If indexing fails, status set to Failed
- Admin notified via dashboard
- Manual re-processing option available

5.3.2 Document Download Flow with Watermarking

1. User Request:

- User clicks “Download” on document
- Laravel checks: document access rules (target audience, user view permissions, user institution) and document active status

2. Watermark Generation:

- Retrieve original document
- Generate unique watermark metadata (user info, timestamp, UUID)
- Embed metadata watermark
- Apply visible text watermark using FPDF (downloader info overlay on each page)

6 | Data Requirements

6.1 Database Schema (High-Level)

An initial set of core tables needed. Detailed schema design (columns, indexes, relationships) to be developed during implementation phase.

6.1.1 Core Tables

users

permissions

roles (Define the roles that can be assigned to users)

role_permissions

user_roles (Defines which roles are assigned to which users)

institutions

documents

document_targets (Polymorphic pivot: links documents to target institutions or roles)

document_versions (Links a document to its previous versions)

tags

`document_tags`
`document_chunks` (RAG embeddings and chunk references)
`download_logs`
`audit_logs`
`notifications`

7 | Security Requirements

7.1 Authorization Requirements

Permission-Based Access Control

- All access decisions based on user's assigned permissions (aggregated from all roles)
- Deny-by-default policy: Explicit permission required for every action
- No privilege escalation possible through application logic

7.2 Data Protection Requirements

Secure Deletion

- Document deletion uses soft-delete: document is marked inactive and visible only to Super-Admin. Data is fully retained in storage.
- Soft-deleted documents can be restored by a Super Admin at any time.

7.3 Application Security

Secure Coding Practices

- All user inputs validated and sanitized (whitelist validation)
- SQL injection prevention via parameterized queries (Eloquent ORM)
- XSS prevention via context-aware output encoding
- CSRF protection: token-based on all state-changing operations
- Rate limiting on authentication endpoints and API routes
- Session security: HttpOnly and Secure cookie flags, idle timeout

8 | Acceptance Criteria

Functional Acceptance

- ☐ Ministry Admin can upload documents with full metadata
- ☐ Uploaded documents visible to authorized users only
- ☐ Document downloads include user-specific watermark metadata
- ☐ Full-text search returns relevant results
- ☐ Security measures enforced for admin accounts
- ☐ 1 pilot university successfully onboarded
- ☐ RAG query interface accepts Arabic natural language questions
- ☐ Query responses include clickable citations to source documents

9 | Future Enhancements

The following features are planned for future releases and are **not included in the initial version**:

1. **Hard Deletion:** Permanent removal of documents from all storage (database and object storage). Would require a dedicated permission restricted to Super Admin.
2. **Logo Watermark Overlay:** Visible logo/image watermark on downloaded documents using Imagick (PHP), supplementing the text-based watermark.
3. **Advanced Invisible Watermarking:** Frequency-domain or canary trapping techniques for enhanced tamper resistance and forensic traceability (see Appendix A).
4. **DOCX Support:** Accept DOCX file format in addition to PDF for document uploads.

Appendix A | Watermarking Strategy & Technical Design

Document Scope

This appendix presents an optional advanced watermarking technique that may supplement the baseline implementation (metadata embedding + visible text overlay, described in Section 3.4). It may be considered as an enhancement after core watermarking is operational.

Every document download must produce a unique, verifiable identifier tied to a specific user. If a copy later appears publicly, forensic analysis must identify the source with certainty.

Appendix A.1 Shared Audit System

All watermarking approaches share the same PostgreSQL audit infrastructure. Every download generates a signed payload stored in a hash-chain table (`watermark_issuances`) where each row incorporates the SHA-256 hash of the previous row. Any tampering breaks the chain and is detected by automated verification.

Listing 1: Watermark payload built at download time

```
{
  "doc_id":      42,
  "user_id":     17,
  "user_name":   "Dr. Yassir Cherdouh",
  "institution": "University of Algiers",
  "email":       "y.cherdouh@univ-alger.dz",
  "ip":          "193.194.80.5",
  "timestamp":   "2026-02-27T15:42:00Z",
  "uuid":        "a3f9c12e-..."
}
```

Appendix A.2 Option 1 - Document Canary Trapping

Option 1 - Document Canary Trapping (Content-Level Fingerprinting)

The fingerprint is embedded inside the **content itself** - not the file's binary structure. Each user receives a textually unique copy with invisible micro-variations: a comma vs. a semicolon, an em dash vs. an en dash, a slightly different line break. The technique is called *canary trapping* and is used by intelligence agencies.

Variations are derived deterministically from the issuance UUID, so the same user always receives the same version. The critical advantage: **this survives retyping** - an

attacker who manually retypes the document unconsciously reproduces the very variations that identify them. There is no binary payload to strip and no pixel to analyse; the identity *is* the text.

With n variation points the system uniquely identifies up to 2^n issuances - 30 well-chosen points cover over one billion distinct copies. The applied bitmask is persisted to `watermark_issuances` alongside the RSA-signed audit record.

Issuing Flow

1. Derive canary bitmask from issuance UUID; select the appropriate variant for each variation point in the document template.
2. Render personalised document with chosen variants; embed signed metadata and apply visible text watermark.
3. Persist bitmask and applied variants to `watermark_issuances`; serve PDF and discard temp files.

Forensic Extraction

1. Extract text from leaked material in any form: PDF, photograph, or OCR of a retyped copy.
2. Compare observed variation pattern against `watermark_issuances`; matching bitmask identifies the issuance and user.
3. A partial match still significantly narrows the candidate set.